

Collaborative Cloud IDE & Sandbox: Complete Engineering Guide

Prepared for Internship Interview Preparation (Semester 5)

1. The Big Picture

The Collaborative Cloud IDE & Sandbox is a high-impact full-stack system designed to address two of the most technically challenging paradigms in modern software engineering: real-time multi-user state synchronization and secure, isolated multi-tenant code execution. Rather than building a generic CRUD application, this project replicates the foundational architecture of industry-grade platforms like VS Code Live Share, Figma, and LeetCode.

In production environments, allowing users to execute arbitrary code on a shared server poses an existential security threat (e.g., malicious scripts deleting system files or initiating infinite loops that exhaust CPU resources). This project showcases your ability to mitigate these risks by containerizing runtime environments dynamically using the Docker Engine API, while simultaneously providing a seamless, low-latency collaborative UI via Conflict-Free Replicated Data Types (CRDTs).

2. Core Features & Capabilities

- **Conflict-Free Real-Time Editing:** Multiple concurrent users can open the same document, write code, and see changes propagate instantly with zero document corruption or text overlapping.
- **Presence & Live Cursors:** Visual indicators mapping the exact cursor positions, text selections, and user profiles of all active collaborators in the workspace.
- **Isolated Code Execution Sandbox:** Support for running multiple language runtimes (e.g., Python, JavaScript, C++) inside transient, highly restricted Docker containers.
- **Streamed Terminal Outputs:** Real-time streaming of stdout, stderr, and execution metrics (compile time, execution duration, and memory usage) back to the client interface.
- **Workspace Persistence:** Secure session management allowing users to generate shareable links and preserve project files across database layers.

3. Technical Architecture & Tech Stack

To maximize architectural stability and high-throughput execution, the system is separated into

decoupled, highly specialized layers:

System Layer	Technology Choice	Engineering Responsibility
Frontend Client	Next.js, TypeScript, TailwindCSS, Monaco Editor	Handles stateful text rendering, abstract syntax highlighting, and immediate client-side optimistic UI feedback.
Synchronization Layer	Yjs, WebSockets (Socket.io), Redis Pub/Sub	Manages document delta distributions via CRDTs and scales WebSocket instances uniformly across multi-server environments.
Orchestration Backend	Node.js (Fastify) or Go (Fiber)	Manages workspace authentication, rooms life cycles, database routing, and secure execution triggers.
Execution Sandbox	Docker Engine API, Linux cgroups	Programmatically provisions unprivileged containers, enforces resource ceilings, and guarantees hard isolation.

4. Step-by-Step Implementation Roadmap

When utilizing AI-assisted tools like Antigravity 2.0 or GitHub Copilot, you must execute this roadmap in highly contained phases to maintain architectural control:

1. **Phase 1: The Monolithic Setup & Editor UI (Days 1–4):** Scaffold the Next.js frontend repository. Embed the Monaco Editor. Set up standard backend routes to save and load code snippets via PostgreSQL or MongoDB.
2. **Phase 2: The Real-Time WebSocket Core (Days 5–9):** Integrate Socket.io. Bind the Monaco Editor state to a Yjs shared document instance over the WebSocket connection.

Verify that typing from one browser instance replicates flawlessly on another without losing character arrays.

3. **Phase 3: The Dockerized Sandbox Engine (Days 10–14):** Write a standalone utility on your backend that interfaces with the Docker daemon. Create a pool of lean base images (e.g., `python:3.10-slim`). Implement a service that takes a code string, writes it to a transient volume, boots a container with `--memory="100m"` and `--cpus="0.5"`, captures output buffers, and immediately deletes the container.
4. **Phase 4: Optimization, Logs Streaming & Security Hardening (Days 15–21):** Wrap the execution process in a hard timeout event wrapper. If code execution exceeds 2000ms, force trigger a container termination signal. Refine the UI to display execution metrics and error diagnostics cleanly.

5. Advanced Engineering Pitfalls & Interview Defenses

Interviewers will drill heavily into the mechanics of this system. Be fully prepared to defend the following design choices:

CRDTs (Yjs) vs. Operational Transformation (OT): Understand why you chose CRDTs over OT. OT requires a centralized, stateful authority server to sequence and recalculate operational transformations continuously. CRDTs (like Yjs) merge document states mathematically, allowing changes to resolve in any order across distributed networks, drastically minimizing server processing overhead.

Mitigating Fork Bombs and Memory Overloading: If a user executes a script designed to consume infinite memory or fork infinite processes (e.g., `while True: os.fork()`), standard backends will crash immediately. Explain how your system uses Docker's native Linux kernel control groups (cgroups) parameters via the API to strictly cap process counts (`--pids-limit=50`) and memory footprints, preventing guest runtimes from starving the host server.